Exp :1 Write the Python Program to solve 8-Puzzle Problem.

Aim:

To write a python program to solve the 8-puzzle problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.

Step2: If the dequeued state is the goal state, return the solution path.

Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its neighboring tiles.

Step5: For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before

Program:

```
import heapq
class PuzzleState:
    def __init__(self, board, moves=0, previous=None):
        self.board = board
        self.moves = moves
        self.previous = previous

    def __lt__(self, other):
        return (self.moves + self.manhattan()) < (other.moves + other.manhattan())

    def manhattan(self):
        dist = 0
        for i in range(3):
            for j in range(3):
                val = self.board[i][j]
```

```
                if val != 0:
                    target_x = (val - 1) // 3
                    target_y = (val - 1) % 3
                    dist += abs(i - target_x) + abs(j - target_y)
        return dist

    def get_successors(self):
        successors = []
        x, y = 0, 0
        for i in range(3):
            for j in range(3):
                if self.board[i][j] == 0:
                    x, y = i, j

        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < 3 and 0 <= ny < 3:
                new_board = [list(row) for row in self.board]
                new_board[x][y], new_board[nx][ny] = new_board[nx][ny], new_board[x][y]
                successors.append(PuzzleState(tuple(tuple(row) for row in new_board), self.moves
+ 1, self))
        return successors

def solve_puzzle(initial_board):
    initial_state = PuzzleState(initial_board)
    goal_board = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

    frontier = []
    heapq.heappush(frontier, initial_state)
    explored = set()
```

```
    while frontier:
        current_state = heapq.heappop(frontier)

        if current_state.board == goal_board:
            path = []
            while current_state:
                path.append(current_state.board)
                current_state = current_state.previous
            return path[::-1]

        explored.add(current_state.board)

        for successor in current_state.get_successors():
            if successor.board not in explored:
                heapq.heappush(frontier, successor)
    return None

if __name__ == '__main__':
    initial = ((1, 2, 3), (4, 0, 5), (7, 8, 6))
    solution = solve_puzzle(initial)

    if solution:
        for step_num, step in enumerate(solution):
            print("Step", step_num, ":")
            for row in step:
                print(row)
            print("")
    else:
        print("No solution found.")
```

OUTPUT:

```
Step 0 :
(1, 2, 3)
(4, 0, 5)
(7, 8, 6)

Step 1 :
(1, 2, 3)
(4, 5, 0)
(7, 8, 6)

Step 2 :
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Result: Hence, the simple python program for 8- puzzle is done